

知能情報基礎演習 IV-2

学籍番号：245709 氏名：神田 聖

実験テーマ名：Web 開発の基礎 担当教員名：宮里 智樹

実験日：2026 年 7 月 27 日 提出期限日：2026 年 7 月 27 日

実際に提出した日：2026 年 7 月 27 日

1 自己紹介 Web ページ

```
1 <!DOCTYPE html>
2 <html lang="ja">
3   <head>
4     <meta charset="UTF-8">
5     <meta name="viewport" content="width=device-width, initial-scale
6   =1.0">
7     <title>私の自己紹介ページ(課題1)</title>
8     <style>
9       /* 簡易的なCSS (見た目をちょっと整える設定) */
10      body {
11        font-family: sans-serif;
12        max-width: 800px;
13        margin: 0 auto;
14        padding: 20px;
15        line-height: 1.6;
16        color: #333;
17        background-color: #f9f9f9;
18      }
19
20      header,
21      main,
22      footer {
23        background-color: #fff;
24        padding: 25px;
25        margin-bottom: 20px;
26        border-radius: 8px;
27        box-shadow: 0 2px 5px rgba(0, 0, 0, 0.05);
28      }
29
30      header {
31        text-align: center;
32        background: linear-gradient(135deg, #2c3e50, #3498db);
33        color: white;
34      }
35
36      h1 {
37        margin-top: 0;
38      }
39
40      h2 {
```

```

40         border-bottom: 2px solid #3498db;
41         padding-bottom: 5px;
42         margin-top: 30px;
43     }
44
45     ul {
46         background-color: #f1f8ff;
47         padding: 20px 20px 20px 40px;
48         border-radius: 5px;
49     }
50
51     .icon-img {
52         display: block;
53         max-width: 100%;
54         height: auto;
55         margin: 10px auto;
56         border: 1px solid #ddd;
57         border-radius: 4px;
58     }
59
60     footer {
61         text-align: center;
62         font-size: 0.9rem;
63         color: #777;
64     }
65 </style>
66 </head>
67
68 <body>
69     <!-- 1. ヘッダー領域 -->
70     <header>
71         <h1>Welcome to My Profile!</h1>
72         <p>HTMLの練習用に作った、私の自己紹介ページです。</p>
73     </header>
74
75     <!-- 2. 自己紹介メインエリア -->
76     <main>
77         <h2>プロフィール</h2>
78         <p>はじめまして！現在、Web開発の基礎を勉強中の神田聖です。</p>
79         <p>2年次の時にこの講義を受講することを忘れていたので、3年次の
今受講することになりました。</p>
80
81         <h2>私の好きなこと</h2>
82         <ul>

```

```

83         <li>物理学（大学院では物理系の研究室を志望しています） </li>
      >
84         <li>音楽を聴くこと</li>
85         <li>勉強すること</li>
86     </ul>
87
88     <h2>自分がよく使っているアイコン</h2>
89     
91     <p>ちなみにこの画像は数式であり、ナビエストークス方程式の移流
92 項という名前の式である。</p>
93
94     <h2>自分が物理を好きになったきっかけの人</h2>
95     <p>
96         通称：ヨビノリ（<a href="https://www.youtube.com/@yobinori
97 " target="_blank">予備校のノリで学ぶ大学の数学・物理のチャンネル</a>）
98     </p>
99     </main>
100
101     <!-- 3. フッター領域 -->
102     <footer>
103         <p>&copy; 2026 神田聖. All Rights Reserved.</p>
104     </footer>
105 </body>
106 </html>

```

Listing 1 課題 1 の html

コード 1 は自己紹介 Web ページである。コード例の方では、少し味気なかったので css で装飾を施した。

Welcome to My Profile!

HTMLの練習用に作った、私の自己紹介ページです。

プロフィール

はじめまして！現在、Web開発の基礎を勉強中の神田聖です。

2年次の時にこの講義を受講することを忘れていたので、3年次の今受講することになりました。

私の好きなこと

- 物理学（大学院では物理系の研究室を志望しています）
- 音楽を聴くこと
- 勉強すること

自分がよく使っているアイコン

$$(\mathbf{u} \cdot \nabla) \mathbf{u}$$

ちなみにこの画像は数式であり、ナビエストークス方程式の移流項という名前の式である。

自分が物理を好きになったきっかけの人

通称：ヨビノリ（[予備校のノリで学ぶ大学の数学・物理のチャンネル](#)）

© 2026 神田聖. All Rights Reserved.

図 1 課題 1 の実験結果

図 1 は課題 1 の実験結果である。

実験結果としては、とても見やすいページとなった。css を使うことで、よりページが鮮やかになることで読みやすくなることが改めて分かる。

2 ランダムに変わる背景色

```
1 <!DOCTYPE html>
2 <html lang="ja">
3   <head>
4     <meta charset="UTF-8">
5     <meta name="viewport" content="width=device-width, initial-scale
6     =1.0">
7     <title> 二つのファイルに分けて連携する（課題2） </title>
8     <script src="script.js" defer></script>
9     <style>
10      /* 見た目の調整 */
11      body { font-family: sans-serif; text-align: center; padding-
12      top: 50px; }
13      button { padding: 10px 20px; font-size: 16px; cursor: pointer;
14      }
15      #layout { font-size: 24px; color: #333; margin-bottom: 20px; }
16    </style>
17  </head>
18  <body> <!-- ボタンの配置 -->
19    <p id="layout">ボタンを押すと、背景の色が変わります</p>
20    <button onclick="change_background_color()">背景色を変える</button
21  >
22  </body>
23 </html>
```

Listing 2 課題2のhtml

```
1 console.log("テスト")
2
3 function change_background_color() {
4   // 0から255までのランダムな整数を作る
5   const r = Math.floor(Math.random() * 256);
6   const g = Math.floor(Math.random() * 256);
7   const b = Math.floor(Math.random() * 256);
8
9   // ページ全体の色を変える
10  document.body.style.background = `rgb(${r}, ${g}, ${b})`;
11 }
```

Listing 3 課題2のjs

コード2はボタンを押すたびに JavaScript で定義されている、change_background_color 関数が呼び出されるようになっている。

コード 3 は r, g, b の値を 0 から 255 個までそれぞれランダムに取得し、背景の色をランダムに変えている。



図 2 課題 2 の実験結果 (ボタンを一回目に押した時の結果)



図 3 課題 2 の実験結果 (ボタンを続けて二回目押した時の結果)

図 2 と 図 3 は課題 1 の実験結果である。これはボタンを押すと背景の色が切り替わったことを意味している。

実験結果としては、意図通りにボタンを押すたびに背景の色が変化した。今回は RGB 空間で指定したが HSV 空間でもできそうだと感じた。特に SV を固定して H をランダム化することで、純粋な色相の変化をさせ変化の分かりやすいようにできると考えた。

3 css を使ってスタイルを当てる

```
1 <!DOCTYPE html>
2 <html lang="ja">
3   <head>
4     <meta charset="UTF-8">
5     <title>自分でスタイルを当ててみよう</title>
6     <link rel="stylesheet" href="style.css">
7   </head>
8   <body>
9     <h1 class="title">自己紹介</h1>
10    <div class="profile-box">
11      <p>はじめまして！Webデザインを勉強中のフロントエンド太郎です</p>
12    </div>
13  </body>
14 </html>
```

Listing 4 課題 3 の html

```
1 .title {
2   color: green;
3   font-size: 32px;
4 }
5
6 .profile-box {
7   background-color: #f0f0f0;
8   padding: 20px;
9 }
```

Listing 5 課題 3 の CSS

コード 4 は、css を使って h1 タグとテキストの背景のレイアウトを施したページとなっている。

コード 5 は h1 タグの色と大きさ、そして背景の色とパディングの広さを定義している。

自己紹介

はじめまして！Webデザインを勉強中のフロントエンド太郎です

図 4 課題 3 の実験結果

図 4 は課題 3 の実験結果である。

実験結果としては h1 タグの色が変わり、パディングを広く取ったことによって灰色帯が太くなっている。パディングを広く取ることで目立ちやすいため、強調させたい時に使えそうだなと考えれる。

4 余白を調整する

```
1 <!DOCTYPE html>
2 <html lang="ja">
3   <head>
4     <meta charset="UTF-8">
5     <title>自分でスタイルを当ててみよう</title>
6     <link rel="stylesheet" href="style.css">
7   </head>
8   <body>
9     <h1 class="title">自己紹介</h1>
10    <div class="profile-box">
11      <p>はじめまして！Webデザインを勉強中のフロントエンド太郎です</
12    p>
13  </div>
14 </body>
</html>
```

Listing 6 課題 4 の html

```
1 .title {
2   color: green;
3   font-size: 32px;
4   margin-bottom: 30px;
5 }
6
```

```

7 .profile-box {
8     background-color: #f0f0f0;
9     padding: 20px;
10    margin-top: 20;
11 }

```

Listing 7 課題 4 の CSS

コード 6 は、課題 3 と同様のため割愛。

コード 5 は、課題 3 の CSS に加えてマージンを設けた。具体的には h1 タグでは下側にマージンを 30px だけ広く設け、テキストの方は上側にマージンを 20px だけ設けた。

自己紹介

はじめまして！Webデザインを勉強中のフロントエンド太郎です

図 5 課題 4 の実験結果

図 5 は課題 3 の実験結果である。

実験結果としては、マージンの相殺ということが起き大きく設定して h1 タグの方のマージンがスタイルとして採用された。ここから、あまり上と下のタグ同士に関係はないと考えることができる。そのため可読性を考えるのならマージンで広げる場合は、常に上で広げるか下で広げるかどうかということを統一した方が良いと言えるであろう。

5 課題 5

5.1 セレクタの取得

```

1 <!DOCTYPE html>
2 <html lang="ja">
3     <head>
4         <meta charset="UTF-8">
5         <title>セレクタの取得</title>
6     </head>
7     <body>
8         <p class="text">一番目の文章</p>

```

```

9      <p class="text target">二番目の文章</p>
10     <script>
11         const el = document.querySelector('p.text.target');
12         console.log(el)
13     </script>
14 </body>
15 </html>

```

Listing 8 課題 5-1 の html

コード 8 は、target が付いた p タグだけ得た情報を入手し、定数 el に代入しコンソール出力しているコードである。

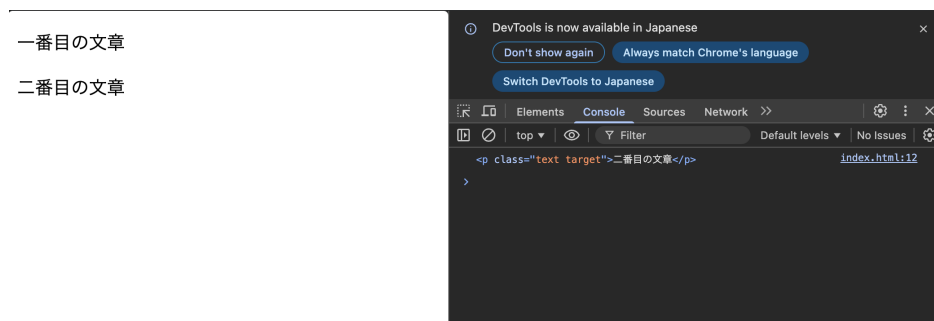


図 6 課題 5-1 の実験結果

図 6 は課題 5-1 の実験結果である。

実験結果としては、出力結果としてはちゃんと target がついたタグだけ入手していることがわかる。これによって差別化をしたい場合はさらに要素を追加することでできる。

5.2 関数の書き換え（アロー関数）

```

1 <!DOCTYPE html>
2 <html lang="ja">
3   <head>
4     <meta charset="UTF-8">
5     <title>関数の書き換え（アロー関数）</title>
6   </head>
7   <body>
8     <button>ボタン</button>
9     <script>
10       const button = document.querySelector('button');
11
12       button.addEventListener('click', () => {
13         console.log('クリックしました')
14       })

```

```

15     </script>
16   </body>
17 </html>

```

Listing 9 課題 5-2 の html

コード 9 は、無名関数でなくてアロー関数を使って、ボタンが押された時にコンソール出力をするようにしている。

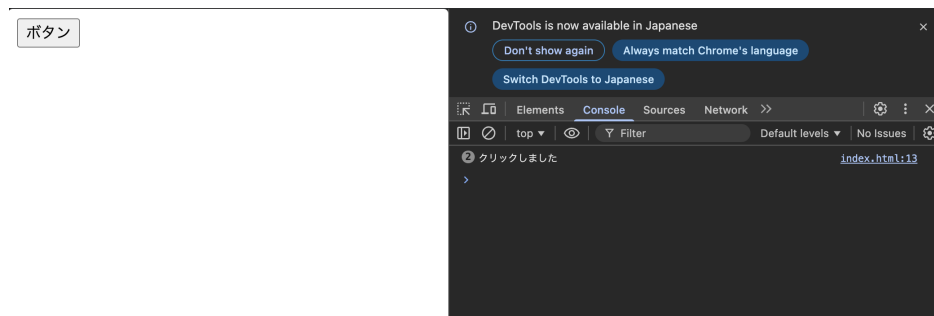


図 7 課題 5-2 の実験結果

図 7 は課題 5-2 の実験結果である。

実験結果としては、ボタンを押すとコメントがコンソール出力された。また、従来の方よりもアロー関数の方が視覚的に分かりやすくなっていると感じる。ただし、使いすぎて入れ子構造になってしまい、逆に分かりづらくなる可能性もあるため、気おつけて使用する必要があるかもしれない。

5.3 アコーディオンメニューの実装

```

1 <!DOCTYPE html>
2 <html lang="ja">
3   <head>
4     <meta charset="UTF-8">
5     <title>アコーディオンメニューの実装</title>
6     <style>
7       .accordion-content {
8         display: none;
9       }
10
11       .accordion-content.active {
12         display: block;
13       }
14
15       .accordion-item {

```

```

16         border: 1px solid #ccc;
17         margin: 10px;
18         padding: 10px;
19         width: 300px;
20     }
21     .accordion-title {
22         cursor: pointer;
23         font-weight: bold;
24         color: #2b7a78;
25     }
26 </style>
27 </head>
28 <body>
29     <div class="accordion-item">
30         <dt class="accordion-title">Q. 返品は可能ですか？</dt>
31         <dd class="accordion-content">A. 未開封に限り、7日以内なら可能
32         です。</dd>
33     </div>
34
35     <script>
36         const title = document.querySelector('.accordion-title');
37
38         title.addEventListener('click', () => {
39             title.nextElementSibling.classList.toggle('active')
40         })
41     </script>
42 </body>
43 </html>

```

Listing 10 課題 5-3 の html

コード 10 は、css で定義された.active を JavaScript で付け外し可能、すなわち表示させるかどうかをトグルで切り替えができるようにすることで、アコーディオンメニュー実現させている。

Q. 返品は可能ですか？

A. 未開封に限り、7日以内なら可能です。

図 8 課題 5-3 の実験結果

図 8 は課題 5-3 の実験結果である。

実験結果としては、.active が取り外し可能なためトグルを使うことで表示と非表示を切り分けれるようになった。これを利用すれば、押したら消えるということやイベントが始めるなど様々なことができると考えられる。

5.4 タブ切り替え UI の実装

```
1 <!DOCTYPE html>
2 <html lang="ja">
3   <head>
4     <meta charset="UTF-8">
5     <title>タブ切り替えUIの実装</title>
6     <style>
7       .tab-group {
8         display: flex;
9         gap: 5px;
10        margin-bottom: 20px;
11      }
12      .tab-btn {
13        padding: 10px 20px;
14        cursor: pointer;
15        background-color: #f0f0f0;
16        border: 1px solid #ccc;
17        border-radius: 4px;
18      }
19      /* アクティブなタブのスタイル */
20      .tab-btn.is-active {
21        background-color: #007bff;
22        color: #fff;
23        border-color: #007bff;
24      }
25    </style>
26  </head>
27  <body>
28    <div class="tab-group">
29      <button class="tab-btn is-active">タブ1</button>
30      <button class="tab-btn">タブ2</button>
31      <button class="tab-btn">タブ3</button>
32    </div>
33
34    <script>
35      const tabs = document.querySelectorAll('.tab-btn');
36
```

```

37     tabs.forEach((clickedTab) => {
38         clickedTab.addEventListener('click', () => {
39             tabs.forEach((tab) => {
40                 tab.classList.remove('is-active')
41             });
42
43             clickedTab.classList.add('is-active')
44         });
45     });
46     </script>
47 </body>
48
49 </html>

```

Listing 11 課題 5-4 の html

コード 11 は、アコーディオンメニューの応用である。ボタンを押した後、一旦全てから `is-active` を付与させないようにしたのちに、押されたボタンだけに対して改めて `is-active` を付与させることにより、タブ切り替え UI を実現されている。



図 9 課題 5-4 の実験結果

図 9 は課題 5-4 の実験結果である。

実験結果としては、最初に `is-active` を全てのボタンに対して剥奪し、また改めて付与しているため、ダブりが存在せずに独立している。これを活用すると、機能の切り替えができるようになる。

6 課題 5 を活用したウェブページ

```

1 <!DOCTYPE html>
2 <html lang="ja">
3     <head>
4         <meta charset="UTF-8">

```

```

5      <title>課題4：タブ切り替えとアコーディオン</title>
6      <link rel="stylesheet" href="style.css">
7  </head>
8
9  <body>
10
11      <div class="container">
12          <!-- 演出①：タブ切り替えUI -->
13          <section class="card">
14              <h2>演出①：タブ切り替えUI</h2>
15              <div class="tab-group">
16                  <button class="tab-btn is-active" data-target="tab1">
お知らせ</button>
17                  <button class="tab-btn" data-target="tab2">サービス</
button>
18                  <button class="tab-btn" data-target="tab3">会社概要</
button>
19              </div>
20              <div class="tab-content-group">
21                  <div id="tab1" class="tab-content is-show">
22                      【2026/07】 本日より、JavaScript×CSS連携講座の募集
を開始いたしました。
23                  </div>
24                  <div id="tab2" class="tab-content">
25                      私たちは、初心者のわかりやすいWeb制作教材・カリ
キュラムを提供しています。
26                  </div>
27                  <div id="tab3" class="tab-content">
28                      東京・渋谷を拠点に、最先端のフロントエンド技術を伝
えるスクールを運営しています。
29                  </div>
30              </div>
31          </section>
32
33          <!-- 演出②：アコーディオンメニュー -->
34          <section class="card">
35              <h2>演出②：アコーディオンメニュー</h2>
36              <div class="accordion-item">
37                  <div class="accordion-title">
38                      <span>Q. プログラミング未経験でも受講できますか？<
/span>
39                      <span class="icon">+</span>
40                  </div>
41                  <div class="accordion-content">

```



```

42         A. はい、大歓迎です。本講座はHTMLの基本を終えたばかり方を対象に設計されています。
43     </div>
44 </div>
45     <div class="accordion-item">
46         <div class="accordion-title">
47             <span>Q. 講義で使用する教材やコードは貰えますか？<
48 /span>
49             <span class="icon">+</span>
50         </div>
51         <div class="accordion-content">
52             A. はい、講義で解説したサンプルコードはすべて終了後に配布いたします。
53         </div>
54     </div>
55 </section>
56 </div>
57     <script src="main.js"></script>
58 </body>
59
60 </html>

```

Listing 12 課題6のhtml

```

1  /* 全体のレイアウト */
2  body {
3      background-color: #f4f5f7;
4      font-family: sans-serif;
5      color: #333;
6      margin: 0;
7      padding: 40px 0;
8  }
9
10 .container {
11     max-width: 600px;
12     margin: 0 auto;
13     display: flex;
14     flex-direction: column;
15     gap: 30px;
16 }
17
18 /* カードのデザイン */
19 .card {
20     background-color: #fff;

```

```

21     border-radius: 8px;
22     box-shadow: 0 2px 8px rgba(0, 0, 0, 0.05);
23     padding: 24px;
24 }
25
26 .card h2 {
27     font-size: 18px;
28     margin-top: 0;
29     margin-bottom: 20px;
30     padding-bottom: 10px;
31     border-bottom: 2px solid #333;
32 }
33
34 /* 演出①：タブ切り替えUIのスタイル */
35 .tab-group {
36     display: flex;
37     border-bottom: 1px solid #ddd;
38     margin-bottom: 16px;
39 }
40
41 .tab-btn {
42     flex: 1;
43     background: none;
44     border: none;
45     padding: 10px;
46     font-size: 14px;
47     cursor: pointer;
48     color: #666;
49     position: relative;
50     text-align: center;
51 }
52
53 .tab-btn.is-active {
54     color: #0066ff;
55     font-weight: bold;
56 }
57
58 .tab-btn.is-active::after {
59     content: '';
60     position: absolute;
61     bottom: -1px;
62     left: 0;
63     width: 100%;
64     height: 2px;

```

```

65     background-color: #0066ff;
66 }
67
68 .tab-content {
69     display: none;
70     font-size: 14px;
71     line-height: 1.6;
72     color: #444;
73 }
74
75 .tab-content.is-show {
76     display: block;
77 }
78
79 /* 演出②：アコーディオンメニューのスタイル */
80 .accordion-item {
81     border: 1px solid #e0e0e0;
82     border-radius: 4px;
83     margin-bottom: 12px;
84     background-color: #f9f9f9;
85 }
86
87 .accordion-title {
88     padding: 14px 16px;
89     font-size: 14px;
90     font-weight: bold;
91     cursor: pointer;
92     display: flex;
93     justify-content: space-between;
94     align-items: center;
95 }
96
97 .accordion-title .icon {
98     font-size: 16px;
99     font-weight: normal;
100 }
101
102 .accordion-content {
103     display: none;
104     padding: 14px 16px;
105     font-size: 14px;
106     background-color: #fff;
107     border-top: 1px solid #e0e0e0;
108     line-height: 1.6;

```

```

109     color: #444;
110 }
111
112 /* アコーディオンが開いたときの状態（クラスが付与されたとき） */
113 .accordion-item.is-open .accordion-content {
114     display: block;
115 }
116
117 .accordion-item.is-open .icon {
118     /* 開いたときに「+」を「×」にするなどの演出 */
119     content: "×";
120 }

```

Listing 13 課題6のCSS

```

1 // =====
2 // 演出①：タブ切り替えUI
3 // =====
4 const tabButtons = document.querySelectorAll('.tab-btn');
5 const tabContents = document.querySelectorAll('.tab-content');
6
7 tabButtons.forEach((button) => {
8     button.addEventListener('click', () => {
9         // すべてのタブボタンから is-active を外す
10         tabButtons.forEach((btn) => {
11             btn.classList.remove('is-active');
12         });
13         // クリックされたボタンに is-active を付与
14         button.classList.add('is-active');
15
16         // すべてのコンテンツから is-show を外して非表示にする
17         tabContents.forEach((content) => {
18             content.classList.remove('is-show');
19         });
20
21         // 該当するコンテンツを表示する
22         const targetId = button.getAttribute('data-target');
23         const targetContent = document.getElementById(targetId);
24         if (targetContent) {
25             targetContent.classList.add('is-show');
26         }
27     });
28 });
29
30 // =====

```

```

31 // 演出②：アコーディオンメニュー
32 // =====
33 const accordionTitles = document.querySelectorAll('.accordion-title');
34
35 accordionTitles.forEach((title) => {
36     title.addEventListener('click', () => {
37         // 親要素 (.accordion-item) のクラスを切り替える
38         const item = title.closest('.accordion-item');
39         item.classList.toggle('is-open');
40
41         // アイコンの文字を「+」と「×」で切り替える
42         const icon = title.querySelector('.icon');
43         if (item.classList.contains('is-open')) {
44             icon.textContent = '×';
45         } else {
46             icon.textContent = '+';
47         }
48     });
49 });

```

Listing 14 課題 6 の js

コード 12 は、Web ページの全体的な構成を記述している。1 段目の section は課題 5 でやったタブ切り替え UI となっており、2 段目の section も課題 5 でやったアコーディオンメニューとなっている。

コード 13 は、Web ページの細かなレイアウトを記述している。ここでは全体的な配置でなく、細かいレイアウトの仕様が定義されている。

コード 14 は主に、課題 5 でやったタブ切り替え UI とアコーディオンメニューが実装されている。一つ新しい他所としては、is-open が付与されている場合にアイコンの文字を変える用になっている。



図 10 課題 6 の実験結果

図 10 は課題 6 の実験結果である。

実験結果はちゃんと課題 6 に満たすような要件が成し遂げている。特に今回は擬似要素という css の要素を使うことにより、それぞれのタブごとに青い下線を引けるようになった。これを使うことで、選択されるとその対象を強調できたりすることができることが言えそうである。

ソースコード